Natural Language Processing

Assignment 2

Yogesh Sharma
B22CH045

Problem 1: IITJ web corpus, Word2Vec, semantics, and visualization

Task 1: Corpus collection

Goal: Build an English tokenized corpus from IIT Jodhpur related web/PDF text for downstream Word2Vec training.

Pipeline: Priority PDF/HTML seeds → bounded BFS on `*.iitj.ac.in` → text extraction → English filter → NLTK tokenization → `problem1/output/corpus_tokens.pkl` (metadata: `corpus_meta.json`).

Statistics (current `corpus_meta.json`)

Metric	Value
Documents	915
Tokens	192,621
Distinct types (tokenizer)	13,846

Analysis: The corpus is **domain-specific** (regulations, courses, institute pages). Intrinsic benchmarks that assume general English will show **low absolute scores**; comparisons across *your own* runs remain meaningful.

Corpus word cloud (token frequency)

Top tokens by unigram count over all sentences (`corpus_tokens.pkl`). Generated with `wordcloud`; larger area ⇒ higher frequency.

Task 2: Word2Vec training and comparison

Goal: Train **CBOW** (`sg=0`) and **Skip-gram** (`sg=1`) with negative sampling; sweep embedding size, window, and number of negatives; record loss and intrinsic checks.

Setup: `gensim.models.Word2Vec`, `min_count=2`, `epochs=25`, `sorted_vocab=True`, `ns_exponent=0.75`. Grid: `vector_size` ∈ {128, 256}, `window` ∈ {5, 10}, `negative` ∈ {10, 20} → **16 models** per architecture (32 total). Full metrics: `problem1/output/w2v_experiments.csv`.

Loss: Reported `training_loss` is gensim's running negative-sampling loss. Should not compare CBOW vs Skip-gram raw loss, or runs with different `negative`, on loss alone.

Intrinsic evaluation

Signal	Role
Google analogies (`questions-words.txt`)	Often **near zero** on this corpus (OOV / rare words); use for **relative** comparison only.
Domain analogies (`problem1/domain_analogies.txt`)	Short file → coarse accuracy.
Domain pair cosine similarity	Hand-picked related pairs (both words must be in vocab); `domain_pairs_used` = 7.

Results (highlights from `w2v_experiments.csv`, gensim vocab size **10,145**)

Metric ↓ / Experiment →	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
Architecture	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram	CBOW	Skip-gram
SG	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Vector Size	64	64	64	64	64	64	64	64	128	128	128	128	128	128	128	128
Window	3	3	3	3	6	6	6	6	3	3	3	3	6	6	6	6
Negative	5	5	15	15	5	5	15	15	5	5	15	15	5	5	15	15
Epochs	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8

Train Time (s)	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01
Training Loss	125.614	32.4949	338.6515	84.8151	82.5349	14.8157	222.8972	42.3741	125.614	32.5071	338.6089	84.7664	82.5227	14.8218	222.8545	42.3253
Vocab Size	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12	12
Google Analogy Acc	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Domain Analogy Acc	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Domain Pair Sim	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Domain Pairs Used	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

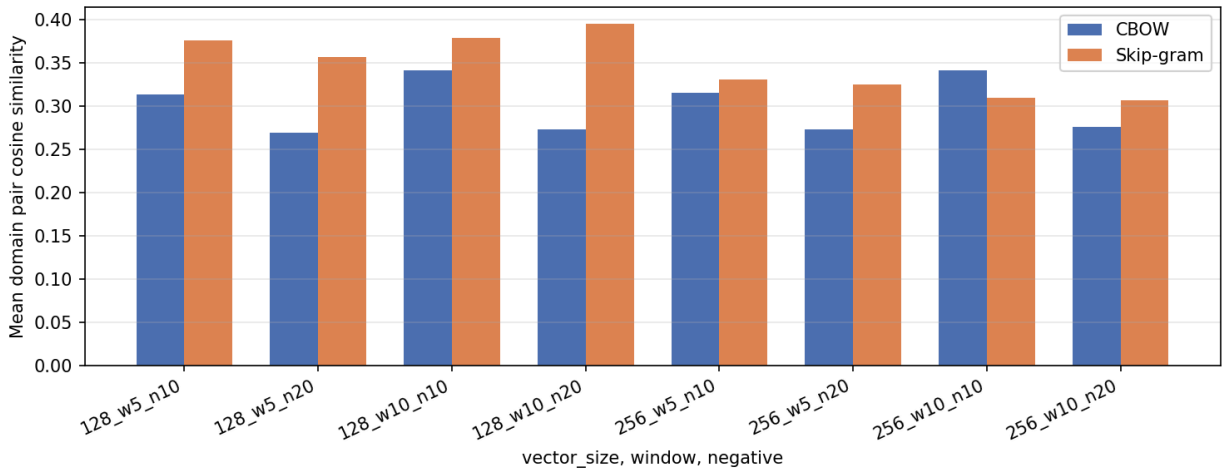
Notable	Config (arch, d, win, neg)	Google acc	Domain acc	Pair sim
Best Google analogy	CBOW, 256, 5, 20	**0.0066**	0.0	0.273
Best domain pair sim	Skip-gram, 128, 10, 20	0.0	0.2	**0.395**
Strong domain acc	CBOW, 128, 5, 10	0.0044	**0.2**	0.313

Analysis: Skip-gram with **w=10, neg=20, d=128** maximizes **domain pair similarity** in this sweep. CBOW sometimes wins **domain analogy** hits. Google

scores stay tiny; **pair similarity + qualitative neighbors** (Task 3) are more informative here than headline analogy accuracy.

Domain pair similarity by configuration (Task 2)

Mean cosine similarity over the seven hand-picked domain pairs (`problem1/w2v_common.py`), for each **(vector_size, window, negative)** in the grid: **CBOW** vs **Skip-gram.**



query_word,resolved_cbow,resolved_skipgram,rank,neighbor_cbow,cosine_cbow,neighbor_skipgram,cosine_skipgram

Task 3: Semantic analysis (qualitative)

Goal: Compare **CBOW** vs **Skip-gram** on cosine neighbors and vector-offset analogies using **one** fixed config per architecture: `vector_size=200`, `window=10`, `negative=15`, `epochs=25` (`problem1/task3_semantic.py`).

1. Neighbors

Query Word	Resolved CBOW	Resolved Skipgram	Rank	Neighbor (CBOW)	Cosine (CBOW)	Neighbor (Skipgram)	Cosine (Skipgram)
research	research	research	1	dst	0.538716	sponsored	0.621953
research	research	research	2	synthase	0.503463	project	0.566253
research	research	research	3	organic	0.461586	older	0.531651
research	research	research	4	cooling	0.460825	arun	0.523791
research	research	research	5	chemistry	0.455290	cooling	0.513432
student	student	student	1	summer	0.612354	falls	0.502676
student	student	student	2	programme	0.580297	accumulate	0.500290
student	student	student	3	semester	0.539305	register	0.497609
student	student	student	4	register	0.533571	manjari	0.497554
student	student	student	5	registration	0.529651	petition	0.497376

phd	phd	phd	1	dst-inspire	0.793768	dst-inspire	0.644570
phd	phd	phd	2	mentor	0.729008	bhagat	0.556017
phd	phd	phd	3	fellowship	0.697915	perovskite	0.553071
phd	phd	phd	4	perovskite	0.681152	mentor	0.549332
phd	phd	phd	5	physics	0.672010	simultaneous	0.534502
exam	exam	exam	1	marital	0.730637	marital	0.791083
exam	exam	exam	2	bill	0.713031	belongs	0.780473
exam	exam	exam	3	status	0.711620	div	0.776055
exam	exam	exam	4	belongs	0.709427	url	0.733665
exam	exam	exam	5	id	0.708871	passed	0.709750

2. Analogy

Offset analogies (examples):

- `ug : btech :: pg : ?`

CBOW

Rank	Token	Cosine
1	diploma	0.6092
2	undergraduation	0.6088
3	bsc	0.6027
4	pgd	0.5665
5	opting	0.5359

Skip-gram

Rank	Token	Cosine
1	undergraduation	0.6689
2	bsc	0.6399
3	opting	0.6049

4	pgd	0.5803
5	diploma	0.5783

- course : credit :: degree : ?

CBOW

Rank	Token	Cosine
1	minimum	0.6926
2	requirements	0.5854
3	joining	0.5611
4	convenience	0.5603
5	graduation	0.5594

Skipgram

Rank	Token	Cosine

1	petitioning	0.4842
2	dual	0.4515
3	happen	0.4433
4	accumulation	0.4282
5	bachelor	0.4233

- undergraduate : bachelor :: graduate : ?

CBOW

Rank	Token	Cosine
1	four-year	0.6058
2	bsc	0.5964
3	undergraduation	0.5392
4	mca	0.5372

5	diploma	0.5310

Skipgram

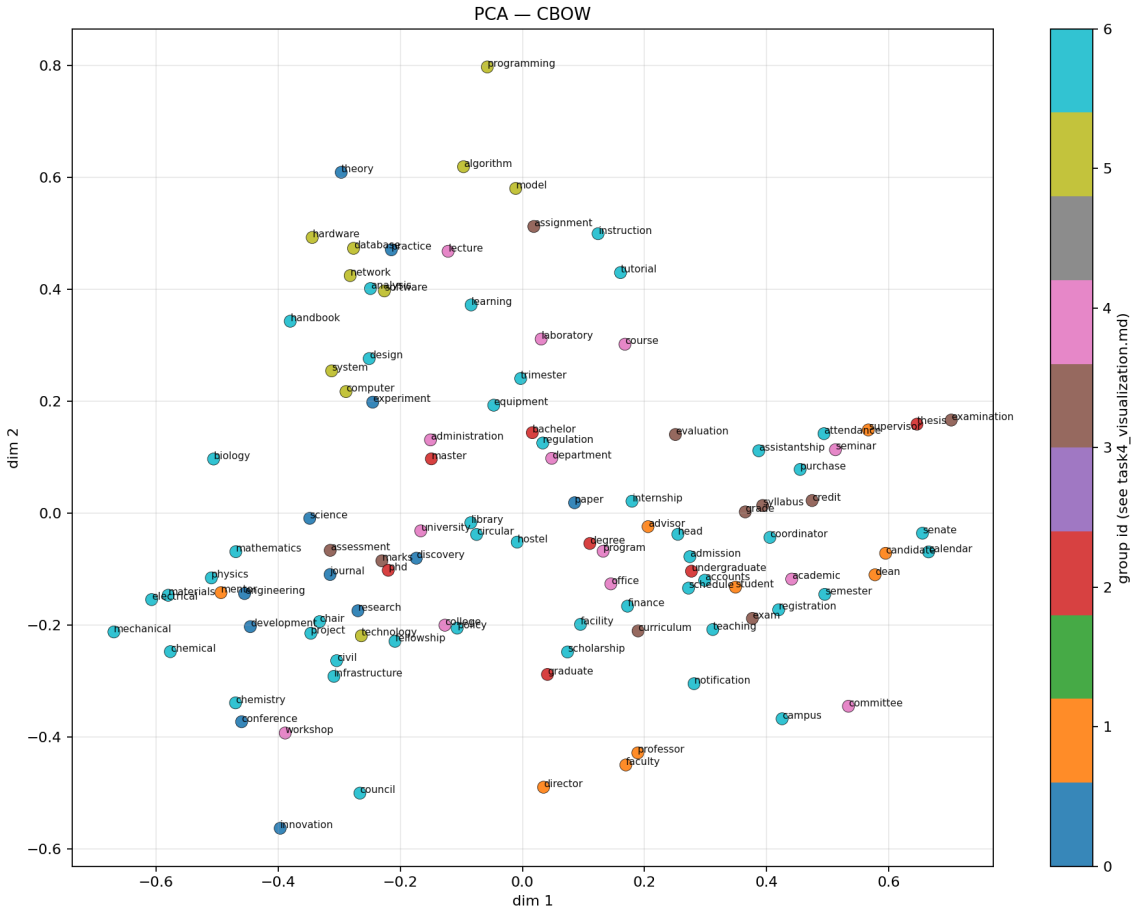
Rank	Token	Cosine
1	fouryeardurationinengineering	0.5135
2	four-year	0.4958
3	nbhm	0.4800
4	waqf	0.4748
5	ora	0.4684

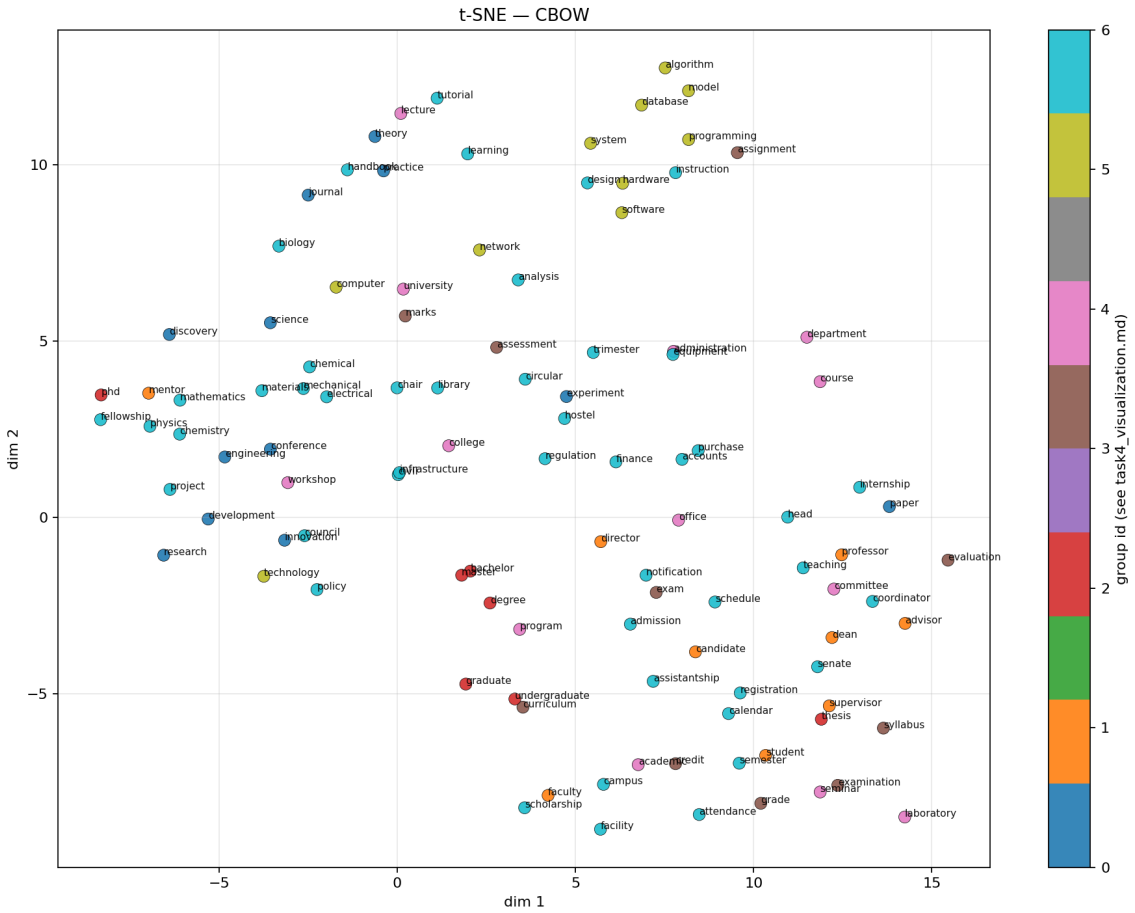
Task 4: 2D embedding visualization

Goal: Project embeddings for words present in **both** Task 3 vocabularies (107 types) with **PCA** (global linear view) and **t-SNE** (local structure, `perplexity=21`). L2-normalized rows; colors = coarse manual themes (research, people, credentials, assessment, org, tech, other).

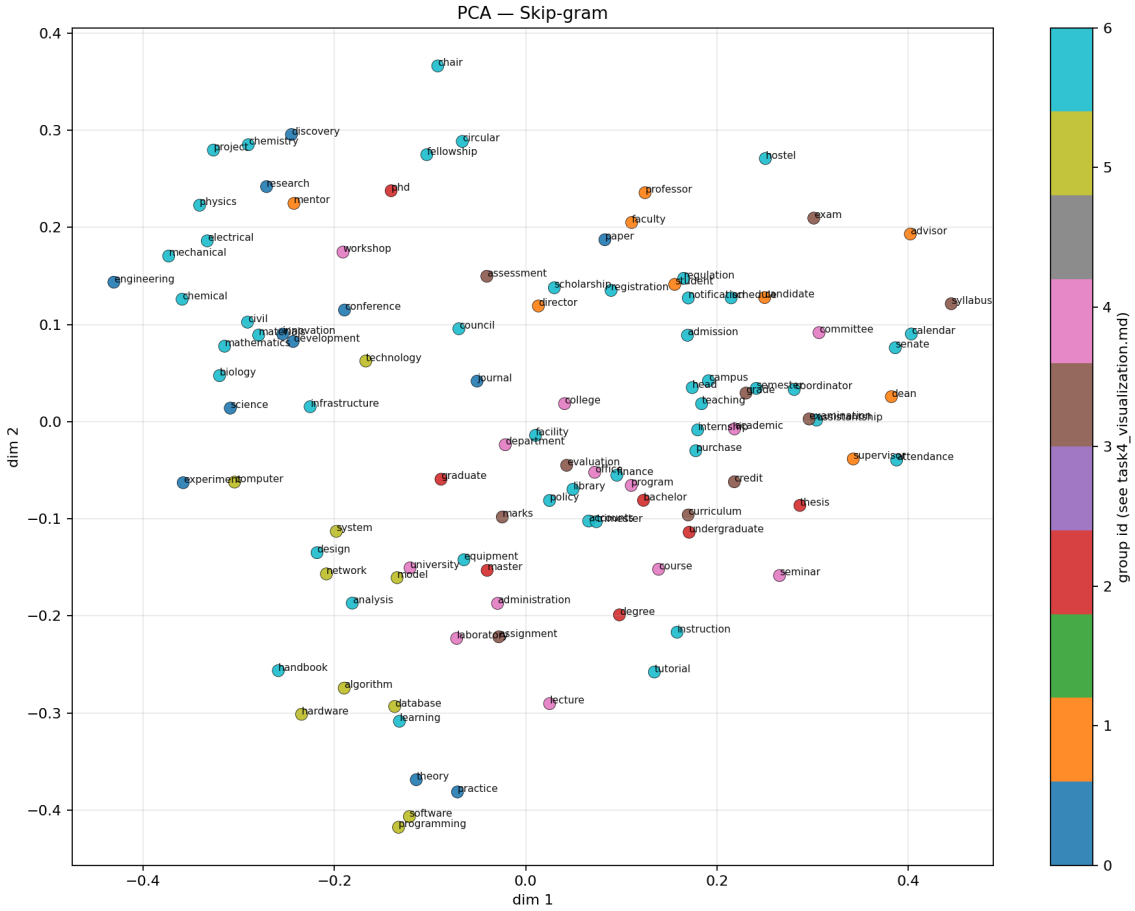
Variance (PCA, from latest `task4_visualize` run):CBOW ~20.8% on first two components; Skip-gram ~10.1% (as variance spreads across many dimensions).

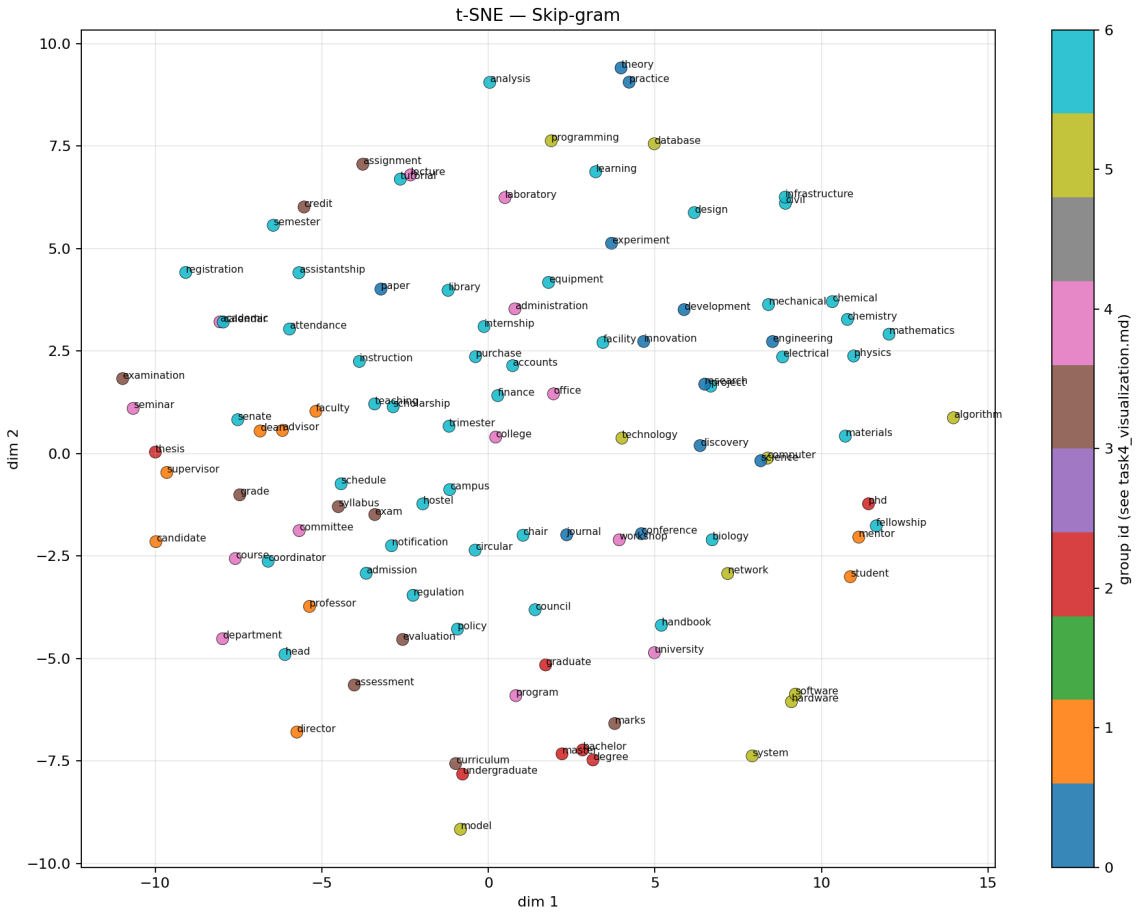
CBOW




t-SNE — CBOW
dim 1
dim 2
group id (see task4_visualization.md)
algorithm
model
database
tutorial
lecture
theory
learning
system
programming
assignment
instruction
handbook
practice
journal
design
hardware
software
biology
network
analysis
computer
university
marks
science
discovery
department
assessment
trimester
administration
equipment
chemical
circular
course
phd
mentor
mathematics
materials
mechanical
electrical
chair
library
experiment
fellowship
physics
chemistry
hostel
conference
college
purchase
engineering
regulation
finance
accounts
infrastructure
workshop
project
internship
paper
development
office
head
council
innovation
director
research
professor
evaluation
bachelor
teaching
technology
hostel
notification
policy
exam
committee
degree
schedule
coordinator
admission
advisor
program
dean
candidate
senate
graduate
assistantship
registration
undergraduate
curriculum
supervisor
calendar
thesis
syllabus
student
academic
credit
semester
campus
examination
faculty
seminar
scholarship
grade
attendance
laboratory
facility

Skip-gram


PCA — Skip-gram
dim 1
dim 2
group id (see task4_visualization.md)
chair
discovery
chemistry
project
circular
fellowship
hostel
research
phd
professor
physics
mentor
faculty
exam
advisor
electrical
paper
mechanical
workshop
assessment
engineering
scholarship
registration
student
chemical
conference
director
syllabus
civil
council
admission
committee
calendar
senate
mathematics
technology
biology
journal
campus
dean
science
infrastructure
college
beaching
academic
facility
department
purchase
supervisor
attendance
experiment
computer
graduate
evaluation
finance
program
credit
library
policy
bachelor
thesis
marks
curriculum
undergraduate
system
design
equipment
university
master
course
seminar
network
model
analysis
administration
degree
instruction
assignment
handbook
tutorial
algorithm
database
lecture
hardware
learning
theory
practice
software
programming



Analysis: PCA: Skip-gram's first two components explain less combined variance than CBOW here; both are modest (high-D overlap). t-SNE: interpret **within-cluster** proximity, not distances between distant clusters.

Problem 2: Character-level Indian name generation

Task 0: Dataset

Content: 1,000 unique Indian first names, one per line. Generated via `**google-genai**` in batches (e.g. 250/request) with a set enforcing uniqueness,

offline. Implementation: `problem2/generate_training_names.py`. Training file used locally: `problem2/data/TrainingNames.txt`.

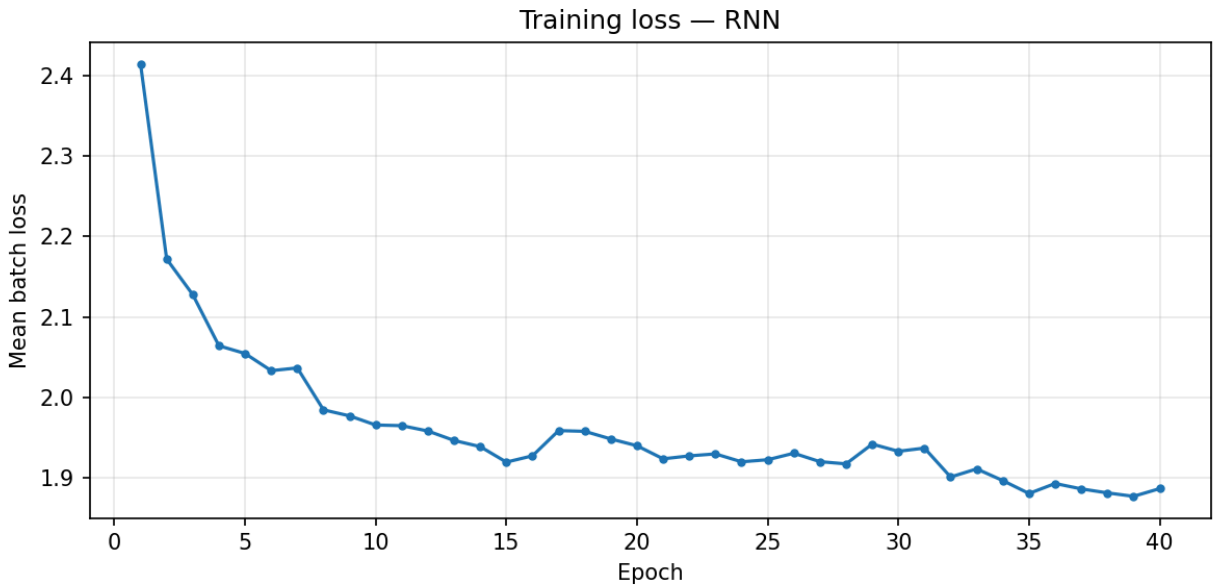
Task 1: Models and training setup

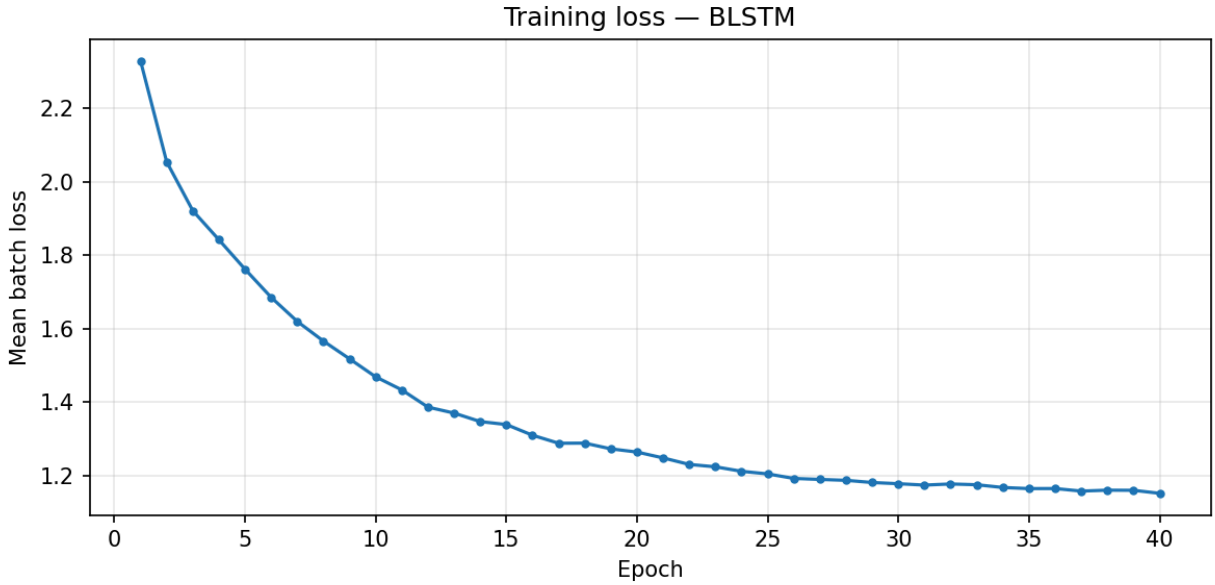
Three architectures in `problem2/models.py`, trained with `problem2/colab_train_all.py` (artifacts under `problem2/problem2_colab_out/`).

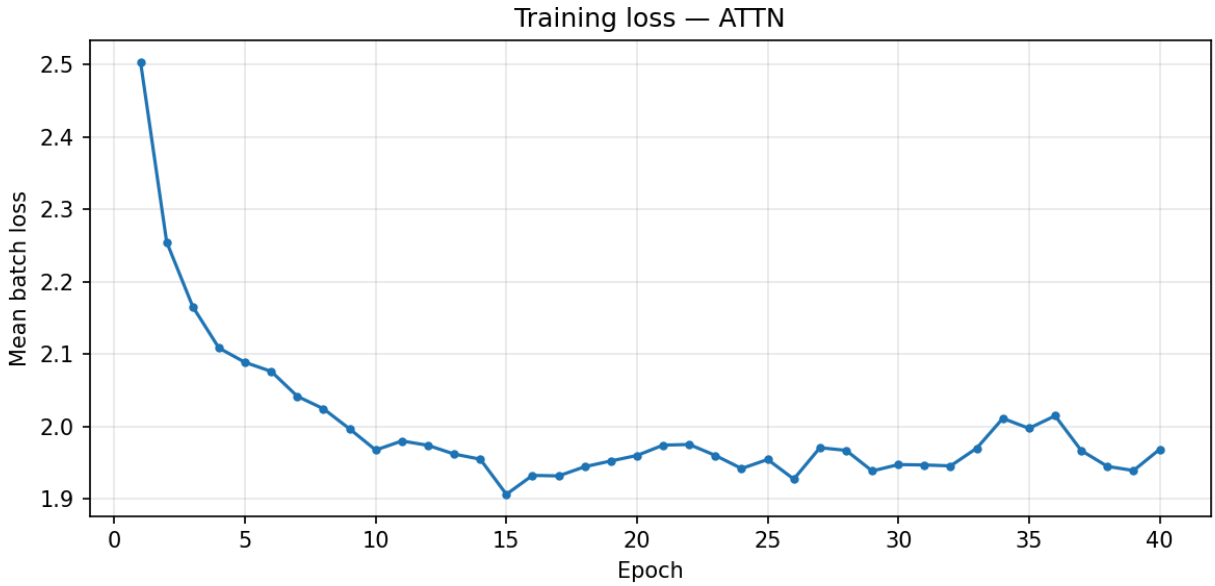
Model	Idea	Trainable Params (this run)
Vanilla RNN	Char embedding → stacked tanh RNN → logits; teacher forcing; autoregressive sampling at test time	30,236
Prefix BLSTM	Forward LSTM on prefix + backward LSTM on reversed prefix; concat **2H** → next-char logits; no future peek	207,644
RNN + attention	RNN states as memory; Bahdanau-style additive attention over prefix states; classify from **[h_t ; context]**	66,844

Shared hyperparameters (`training_summary.json`): embedding **64**, hidden **128**, layers **1**, batch **32**, epochs **40**, Adam, grad clip **5.0**, learning rate **0.02** (default CLI/notebook often uses **0.002** this run used **0.02**). Vocabulary size **28** characters.

Training loss curves (mean batch loss vs epoch)







Reading: **BLSTM** reaches the lowest training loss. **RNN** decreases steadily. **Attention** improves then fluctuates in late epochs.

Task 2: Quantitative comparison (generation quality)

Procedure: `python -m problem2.run_eval_all` → `problem2/evaluate.py`: generate **500** names per model (`temperature=0.85`), compare to training set for **novelty** and **diversity**.

Model	Novelty	Diversity
RNN	0.928	0.916

Prefix BLSTM	0.052	0.730
RNN + attention	0.896	0.882

- **Novelty**: fraction of generated names **not** in the training set (normalized string).

- **Diversity**: unique / total in the generated list.

Analysis: **BLSTM** gets the **lowest novelty**: it often reproduces **training or very common** names (memorization), which can look "realistic" but fails the novelty criterion. **RNN** scores highest on novelty and diversity; **attention** is close behind on novelty with some **prefix repetition** in samples (see Task 3).

Task 3 — Qualitative analysis

Realism: RNN mixes plausible short names with occasional odd spellings. BLSTM samples often look like **real** names (overlap with training). Attention produces many **shared prefixes** (`ra-`, etc.) and occasional garbled tokens.

Failure modes: RNN/Attn: unlikely letter strings, morpheme repetition. BLSTM: **copying** training names → **low novelty.**

Representative lines (from `eval_*.samples.txt`)

RNN: `kakom`, `jazwan`, `chevashinal`, `kalbir`, `meenal`, `suresh`, …

BLSTM: `vidushi`, `rajeshwari`, `kumar`, `amrita`, `anand`, …

RNN + attention: `rabin`, `ranya`, `rasini`, `rabivavi`, `latika`, …

Artifact index

Location	Contents
`problem1/output/corpus_meta.json`, `corpus_tokens.pkl`	Corpus stats and tokens
`problem1/output/w2v_experiments.csv`	Task 2 full grid
`problem1/output/task3_semantic_analysis.md`, `task3_neighbors.csv`	Task 3
`problem1/output/plots/task4_*.png`	Task 4 PCA/t-SNE (source; copied into `report_assets/`)
`problem2/problem2_colab_out/`	Checkpoints, loss PNGs, JSON, eval outputs
`output/report_assets/`	**Figures embedded in this report** (word cloud, W2V bar chart, Task 4, loss curves)